

React Frontend - Part 1 (Static UI with Mock Data) - Problem Statement

Overview

In this week, you will build the first version of the EasyMart React frontend using Vite and plain React. The application will use **mock product data** (no backend calls yet). You will learn React fundamentals: components, props, state (**useState**), event handling, conditional rendering, and client-side filtering/pagination. Navigation is managed with a simple **currentPage** state and the browser's History API (no React Router yet).

Learning Objectives

By the end of this week, you will be able to:

- Scaffold a React + Vite project
- Build reusable functional components with props
- Manage local component state with **useState**
- Conditionally render different views based on state
- Implement client-side search, filter, sort, and pagination
- Build a login/register form with client-side validation
- Display a product detail page with quantity controls and a review form
- Show a Notification/toast component
- Implement hash-based navigation with the browser History API
- Organize a React project into **components/**, **pages/**, **data/**, and **utils/** folders

Required Project Structure

You must create the following exact folder structure:

```
├── index.html
├── package.json
├── vite.config.js
├── src/
│   ├── main.jsx                # React entry point
│   └── App.jsx                 # Root component – state +
navigation
│   ├── App.css
│   └── assets/
│       └── logo.svg            # EasyMart logo (SVG)
```

```

├── data/
│   └── products.js                # MOCK_PRODUCTS array (10
products)
├── utils/
│   ├── constants.js             # CATEGORIES array
│   └── helpers.js               # formatPrice, formatDate,
validateEmail,
├── components/
│   └── layout/
│       ├── Navbar.js           # Top nav: Home / Products /
Login links
│   ├── Navbar.css
│   ├── Footer.jsx
│   └── Footer.css
├── common/
│   ├── Notification.jsx        # Success / error / info toast
│   └── Notification.css
├── products/
│   ├── ProductCard.jsx        # Single product card (image,
name, price, rating)
│   ├── ProductCard.css
│   └── ProductList.jsx        # Grid of cards + search bar +
pagination
│   ├── ProductList.css
│   ├── ProductSearch.jsx      # Search input, category, sort,
price range, in-stock
│   ├── ProductSearch.css
│   └── ProductDetail.jsx      # Full product page: image, info,
cart btn, reviews
│   ├── ProductDetail.css
│   └── Pagination.jsx        # Previous / page numbers / Next
buttons
│   └── Pagination.css
├── components/auth/
│   ├── LoginForm.jsx          # Tabbed Login + Register with
client-side validation
│   └── LoginForm.css
├── pages/
│   ├── HomePage.jsx          # Hero banner + Featured Products
grid
│   └── HomePage.css

```

Setup Instructions

Step 1: Scaffold the project

```
npm create vite@latest EasyMart-Frontend -- --template react
cd EasyMart-Frontend
npm install
```

Step 2: Run the dev server

```
npm run dev
```

The app will be available at <http://localhost:3000>.

Tasks Overview

Part 1: Project Foundations

1. Create **MOCK_PRODUCTS** data array
2. Create utility helpers (**formatPrice**, **validateEmail**, etc.)
3. Create **CATEGORIES** constant

Part 2: Layout Components

4. Build **Navbar** — hash-based navigation, shows user name + Logout when logged in
5. Build **Footer**

Part 3: Common Components

6. Build **Notification** — auto-dismisses, supports **success**, **error**, **info** types

Part 4: Product Components

7. Build **ProductCard** — clickable card with image, name, price, star rating, stock badge
8. Build **ProductSearch** — search input + category dropdown + sort dropdown + price range + in-stock checkbox + **Clear Filters** button
9. Build **Pagination** — disabled Prev / page numbers / Next
10. Build **ProductList** — combines **ProductSearch**, product grid, **Pagination**, and "no results" message
11. Build **ProductDetail** — full detail view: image, category badge, name, rating, price, description, stock info, quantity stepper, **Add to Cart** button, review list, **Write a Review** form

Part 5: Auth Component

12. Build **LoginForm** — tabbed component (Login tab + Register tab) with field-level validation and show/hide password toggle. On success, calls **onLogin** prop with `{ name, email }` (no real API — simulated).

Part 6: Pages & App

13. Build **HomePage** — hero section with **Shop Now** CTA + featured products grid
14. Wire everything together in **App.jsx**:
- Hash-based navigation (**currentPage** state: `'home', 'products', 'login', 'detail'`)
 - Auth state (**isLoggedIn**, **user**)
 - All filter state for the products page
 - **showNotification** helper
 - **navigate(page, product?)** updates state **and** **history.pushState**
 - **popstate** listener for browser back/forward

Part 1: Project Foundations

Task 1: Mock Product Data (**data/products.js**)

Create and export a **MOCK_PRODUCTS** array of **10 product objects**. Each product must have the following shape:

```
{
  _id: String,           // e.g. '1'
  name: String,
  description: String,
  price: Number,
  category: String,      // one of CATEGORIES
  stock: Number,
  rating: Number,        // 0-5
  numReviews: Number,
  images: [String],      // array with one Unsplash URL
  isFeatured: Boolean,   // optional – used for the home page
}
```

Include products from at least **5 different categories** (Electronics, Clothing, Books, Home, Sports, Toys, Other).

Task 2: Utility Helpers (**utils/helpers.js**)

```
// Format a number as USD currency
export const formatPrice = (price) => { ... };
```

```
// Format a date string as "Month DD, YYYY"
export const formatDate = (date) => { ... };

// Basic email regex check – returns Boolean
export const validateEmail = (email) => { ... };

// Extract a human-readable error message from an Axios error or plain
Error
export const getErrorMessage = (error) => { ... };

// Truncate text to maxLength chars and append "..."
export const truncateText = (text, maxLength) => { ... };
```

Task 3: Constants (`utils/constants.js`)

```
export const CATEGORIES = [
  'Electronics', 'Clothing', 'Books', 'Home', 'Sports', 'Toys', 'Other',
];
```

Part 2: Layout Components

Task 4: `Navbar`

Props: `currentPage`, `setCurrentPage`, `isLoggedIn`, `user`, `onLogout`

Requirements:

- Display the **EasyMart** logo and brand name (clicking navigates to `'home'`)
- Navigation links: **Home**, **Products**, **Login** (highlight the active page)
- When `isLoggedIn` is `true`: show `"Hello, {user.name}"` and a **Logout** button instead of the Login link
- Use `<button>` elements (no `<a>` tags — navigation is state-based)

Task 5: `Footer`

A simple footer with the copyright line: © `{year}` EasyMart. All rights reserved.

Part 3: Common Components

Task 6: `Notification`

Props: `message`, `type` (`'success'` | `'error'` | `'info'`), `onClose`

Requirements:

- Render a coloured banner at the top of the main content area
 - Auto-dismiss after **3 seconds** using `useEffect` + `setTimeout`
 - Include a manual close (×) button that calls `onClose`
 - Apply a different CSS class/colour for each `type`
-

Part 4: Product Components

Task 7: `ProductCard`

Props: `product`, `onSelect`

Requirements:

- Display product image (with a fallback URL on `onError`)
- Show an **"Out of Stock"** badge when `stock === 0`
- Show name, formatted price, star rating (★), and review count
- Show a coloured "In Stock" / "Out of Stock" status text
- The entire card is clickable and calls `onSelect(product)`

Task 8: `ProductSearch`

Props: `searchTerm`, `setSearchTerm`, `selectedCategory`, `setSelectedCategory`, `sortBy`, `setSortBy`, `minPrice`, `setMinPrice`, `maxPrice`, `setMaxPrice`, `inStockOnly`, `setInStockOnly`

Requirements:

- Text input for keyword search
- `<select>` for category filtering (from `CATEGORIES` constant + "All Categories" option)
- `<select>` for sort order with options:
 - Default
 - Price: Low to High
 - Price: High to Low
 - Highest Rated
 - Name: A–Z
- Min price and max price number inputs
- "In Stock Only" checkbox
- A **Clear Filters** button that resets all filters (only visible when at least one filter is active)

Task 9: `Pagination`

Props: `currentPage`, `totalPages`, `onPageChange`

Requirements:

- Render **Previous** and **Next** buttons (disabled at first/last page)
- Render individual page-number buttons
- Highlight the currently active page number

Task 10: `ProductList`

Props: all filter props (passed through to `ProductSearch`), `products`, `totalProducts`, `currentPage`, `totalPages`, `onPageChange`, `onSelectProduct`

Requirements:

- Page header: "Our Products" + "{n} product(s) found"
- Render `<ProductSearch>` with all filter props
- If `products` is empty: show a "No products match your search" message
- Otherwise render a CSS grid of `<ProductCard>` components
- Render `<Pagination>` at the bottom

Task 11: `ProductDetail`

Props: `product`, `isLoggedIn`, `showNotification`

Requirements:

- Large product image with fallback; overlay "Out of Stock" if stock is 0
- Category badge, product name, star rating row, formatted price, description
- Stock info line ("In Stock (N available)" vs "Out of Stock")
- If in stock:
 - Quantity stepper (– / number / +) clamped to `[1, stock]`
 - **Add to Cart** button → if not logged in: `showNotification('Please log in...', 'error')`; if logged in: `showNotification('{name} (xN) added to cart!')`
- Reviews section:
 - List of submitted reviews (starts empty) — each shows author name, star rating, comment, date
 - If logged in and has not reviewed yet: **Write a Review** toggle button
 - Review form: star rating `<select>` (1–5) + comment `<textarea>` + **Submit** button
 - On submit: validate comment is not empty, add review to local state, set `alreadyReviewed = true`

Part 5: Auth Component

Task 12: `LoginForm`

Props: `onLogin`, `onCancel`, `showNotification`

Requirements:

- Two tabs: **Login** and **Register** (switch between them; switching resets errors)

- **Login tab fields:** Email, Password (show/hide toggle)
 - Validation: email required + valid format, password required + min 6 chars
 - On success: call `onLogin({ name: email.split('@')[0], email })`
- **Register tab fields:** Full Name, Email, Password (show/hide), Confirm Password (show/hide)
 - Validation: name required, email required + valid, password min 6 chars, passwords must match
 - On success: call `onLogin({ name, email })`
- Show field-level error messages below each invalid input
- A **Cancel / Back** button that calls `onCancel`

Note: This is a simulated login — no real API calls. Next week will replace this with real JWT authentication.

Part 6: Pages & App

Task 13: `HomePage`

Props: `products` (featured/first N products), `onSelectProduct`, `onShopNow`

Requirements:

- Hero section: heading "Welcome to EasyMart", subtitle, **Shop Now** button (calls `onShopNow`)
- Featured Products section: `<h2>Featured Products</h2>` + a grid of `<ProductCard>` components

Task 14: Wire up `App.jsx`

`App.jsx` is the single source of truth for all application state.

State to manage:

State variable	Type	Description
<code>currentPage</code>	<code>string</code>	<code>'home' 'products' 'login' 'detail'</code>
<code>selectedProduct</code>	<code>object \ null</code>	The product to show in detail view
<code>isLoggedIn</code>	<code>boolean</code>	Auth status
<code>user</code>	<code>object \ null</code>	<code>{ name, email }</code>
<code>notification</code>	<code>object \ null</code>	<code>{ message, type }</code>
<code>searchTerm</code>	<code>string</code>	
<code>selectedCategory</code>	<code>string</code>	
<code>sortBy</code>	<code>string</code>	
<code>minPrice</code>	<code>string</code>	

State variable	Type	Description
maxPrice	string	
inStockOnly	boolean	
pageNum	number	Current page (1-based)

navigate(page, product?) helper:

- Sets `currentPage` and `selectedProduct`
- Calls `history.pushState({ page }, '', '#' + urlHash)` (for 'detail', keep the previous URL hash)

Initial page from URL hash:

```
const [currentPage, setCurrentPage] = useState(() => {
  const hash = window.location.hash.slice(1);
  return VALID_PAGES.includes(hash) ? hash : 'home';
});
```

popstate listener (browser back/forward):

```
useEffect(() => {
  if (!window.location.hash) history.replaceState({ page: 'home' }, '', '#home');
  const handlePopState = (e) => { /* restore page from e.state or hash */ };
  window.addEventListener('popstate', handlePopState);
  return () => window.removeEventListener('popstate', handlePopState);
}, []);
```

Client-side filtering pipeline (applied to `MOCK_PRODUCTS`):

1. Filter by `searchTerm` (name or description, case-insensitive)
2. Filter by `selectedCategory`
3. Filter by `minPrice` and `maxPrice`
4. Filter by `inStockOnly`
5. Sort by `sortBy`
6. Paginate: slice to `ITEMS_PER_PAGE = 8` items per page

Page rendering (`renderPage`):

- 'login' → <LoginForm>
- 'detail' + selectedProduct → <ProductDetail>
- 'products' → <ProductList> (with all filter props and paginated products)
- default ('home') → <HomePage> (show **isFeatured** products, or first 3 as fallback)

JSX structure:

```
<div className="App">
  <Navbar ... />
  <main className="main-content">
    {notification && <Notification ... onClose={() =>
setNotification(null)} />}
    {renderPage()}
  </main>
  <Footer />
</div>
```

What You Can Verify in the Browser

Run `npm run dev` and open `http://localhost:3000`. Use this guide to check every feature area.

Important scope notes for this week:

- There is **no real Cart page** — "Add to Cart" only shows a notification. A real cart comes in later weeks.
- There is **no real authentication** — login accepts any valid-format email/password and stores the name in state. Real JWT auth comes in the backend week and the final frontend week.
- Navigation uses URL **hashes** (e.g. `#products`) not real paths — React Router comes in a later week.

Home Page

Action	Expected result
Open <code>http://localhost:3000</code>	Home page with hero banner and "Featured Products" section
Click Shop Now button	Navigates to Products page
Click any featured product card	Product detail view opens

Products Page & Filtering

Action	Expected result
--------	-----------------

Action	Expected result
Click Products in navbar	Grid of products (8 per page)
Type a keyword in the search box	Product list filters in real time
Select a category from the dropdown	Only products in that category show
Select "Price: Low to High" from sort	Products reorder cheapest first
Enter a min price (e.g. 20)	Products below that price disappear
Enter a max price (e.g. 100)	Products above that price disappear
Check In Stock Only	Out-of-stock products disappear
Click Clear Filters (appears when any filter is active)	All filters reset, full list returns
No products match filters	"No products match your search" message (not blank)

Pagination

Action	Expected result
Page 1 (default)	First 8 products shown; Prev button disabled
Click 2 or Next	Next set of products shown
Last page	Next button disabled
Current page number	Visually highlighted

Product Detail

Action	Expected result
Click any product card	Full detail page: image, name, price, category badge, rating, description
Click – below quantity 1	Button disabled; quantity stays at 1
Click + up to max stock	Quantity increments up to stock value, then stops
Click Add to Cart (not logged in)	Red error notification: "Please log in to add items to your cart."
Click Add to Cart (logged in)	Green success notification: "{Product} (xN) added to cart!"
Click Write a Review (logged in)	Review form appears with rating and comment fields

Action	Expected result
Submit review with empty comment	Validation error shown
Submit valid review	Review appears in list; Write a Review button hidden

Authentication

Action	Expected result
Click Login in navbar	Login/Register form appears
Submit Login with empty fields	Field-level error messages under each input
Submit Login with invalid email format	Email error message
Submit Login with password shorter than 6 chars	Password error message
Submit Login with valid data (any email/password)	Logged in; navbar shows "Hello, {name}" + Logout button
Click Register tab	Register form with Name, Email, Password, Confirm
Submit Register with mismatched passwords	"Passwords do not match" error
Submit Register with valid data	Logged in as the new name
Click Logout	Auth state cleared; navbar reverts to Login link

Notifications

Action	Expected result
Trigger any action (login, add to cart)	Coloured toast appears at top of page
Wait 3 seconds	Toast auto-dismisses
Click ✕ on toast	Toast dismisses immediately

Navigation

Action	Expected result
Click Home / Products / Login in navbar	Correct page renders; URL hash updates (e.g. #products)
Press browser Back button	Previous page shown
Press browser Forward button	Returns to the page you went back from

